LEAD University
Sistemas Operativos – IIIC 2026
Bachillerato en Ciencia de Datos

Tarea Programada 2
MiniOS: Planificación de procesos con Round Robin
Duración: 2 semanas (del 12 al 26 de octubre de 2026)

Fecha de Entrega: 26 de Octubre de 2026 antes de la media noche.

1. Marco Contextual

En la Tarea Programada 1 se construyó MiniOS, un simulador de un computador con un único procesador que atendía los procesos en el orden en que llegaban (FIFO) y sin interrumpirlos. Esa política es sencilla y respeta el orden de llegada, pero tiene una limitación importante: un proceso largo que llega primero puede hacer esperar mucho tiempo a procesos cortos que llegaron después. Este fenómeno se conoce como *efecto convoy*.

Los sistemas operativos de tiempo compartido resuelven este problema permitiendo que el sistema operativo le quite el procesador a un proceso antes de que termine para entregárselo a otro. Esto se llama *expropiación* (*preemption*). Gracias a ella, cada proceso avanza de forma periódica y el usuario percibe que varios programas se ejecutan "al mismo tiempo", aunque exista un único CPU.

Round Robin (RR) es uno de los algoritmos de planificación expropiativa más conocidos. En esta segunda tarea se extenderá MiniOS para incorporarlo y se compararán sus resultados con los de FIFO. Con ello se estudiarán conceptos como

- quantum (*time slice*);
- expropiación;
- cambios de contexto;
- tiempo de respuesta;
- utilización del CPU;
- relación entre el tamaño del quantum y el rendimiento del sistema;
- comparación entre políticas de planificación.

La tarea tiene una duración de dos semanas porque, además de implementar el algoritmo, se espera que el estudiante reorganice su código, pruebe casos límite y analice los resultados obtenidos.

2. Objetivo general

Extender el simulador **minios.py** desarrollado en la Tarea Programada 1, de manera que, además de la política FIFO, permita simular la administración de procesos mediante el algoritmo **Round Robin**, calculando métricas y generando archivos de resultados que permitan comparar ambas políticas.

El programa continuará representando un computador con **un único procesador**, pero ahora el CPU podrá ser retirado a un proceso que todavía no ha terminado.

En esta segunda versión existirán dos políticas, que se seleccionarán al ejecutar el programa:

- **FIFO:** igual que en la Tarea 1. Una vez que un proceso obtiene el CPU, lo conserva hasta terminar.
- **Round Robin:** cada proceso puede usar el CPU durante un máximo de Q unidades de tiempo consecutivas (el *quantum*). Si no ha terminado al agotarse el quantum, pierde el CPU y vuelve al final de la cola de procesos listos.

3. Objetivos específicos

Al finalizar la tarea, el estudiante deberá ser capaz de:

1. explicar los conceptos de expropiación y de quantum;
2. implementar el algoritmo Round Robin utilizando una cola de procesos listos;
3. modelar el tiempo restante de ejecución de cada proceso;
4. manejar correctamente los casos en que un proceso llega en el mismo instante en que otro agota su quantum;
5. calcular métricas de espera, retorno, respuesta, utilización del CPU y cambios de contexto;
6. comprender por qué la fórmula del tiempo de espera de la Tarea 1 deja de ser válida cuando los procesos pueden ser interrumpidos;
7. recibir parámetros de configuración desde la línea de comandos;
8. ejecutar experimentos con diferentes valores de quantum y comparar Round Robin contra FIFO;
9. organizar el código en funciones reutilizables y verificarlo con casos de prueba propios.

Debe continuar desarrollando el programa llamado: **minios.py** (el mismo de la Tarea 1, ahora extendido). La política FIFO deberá seguir funcionando.

El programa deberá simular el comportamiento de un computador que posee:

- un único CPU;
- una cola de procesos listos;
- procesos que llegan en diferentes momentos;
- con Round Robin, ejecución interrumpida cuando se agota el quantum;
- con FIFO, ejecución no interrumpida del proceso actual.

4. Escenario

Imagine nuevamente un servidor utilizado para ejecutar tareas relacionadas con procesamiento de datos, como cargar un conjunto de datos, transformar registros o calcular estadísticas. Ahora el servidor es compartido por varias personas analistas y se desea que ninguna tarea corta quede esperando demasiado detrás de una tarea larga.

Cada una de estas tareas será representada como un **proceso**, con las mismas tres características de la Tarea 1:

- un identificador;
- un instante de llegada al sistema;
- una cantidad de tiempo de CPU necesaria para completar su ejecución.

Por ejemplo:

Proceso	Llegada	Tiempo requerido de CPU
P1	0	5
P2	2	3
P3	3	8
P4	6	2

Además, el sistema operativo tendrá un parámetro de configuración: el **quantum**, que es la cantidad máxima de unidades de tiempo que un proceso puede usar el CPU de forma consecutiva. Suponga un quantum de 3. Esto significa, por ejemplo, que:

- P1 llega en el instante 0 y obtiene el CPU de inmediato;
- en el instante 3 P1 habrá usado todo su quantum, pero todavía le faltarán 2 unidades de tiempo;
- como P2 y P3 están esperando, P1 pierde el CPU y pasa al final de la cola;
- P1 volverá a usar el CPU cuando le llegue su turno.

5. Parámetros de ejecución

El programa deberá recibir su configuración desde la línea de comandos (puede utilizar el módulo argparse). No debe solicitar datos al usuario mediante input().

Ejemplo de ejecución:
python minios.py --archivo procesos.csv --politica rr --quantum 3

Parámetro	Significado	Valor por defecto
--archivo	Ruta del archivo CSV con los procesos.	procesos.csv
--politica	Política de planificación: fifo o rr.	rr
--quantum	Entero mayor que cero. Solo se utiliza con la política rr; con fifo se ignora.	3

El programa debe poder ejecutarse también sin ningún parámetro, utilizando los valores por defecto. Si se indica una política desconocida o un quantum inválido, deberá mostrarse un mensaje comprensible y terminar sin ejecutar la simulación.

6. Archivo de entrada

El formato del archivo de entrada **no cambia** respecto a la Tarea 1. Los procesos deberán cargarse desde un archivo CSV; como mínimo se trabajará con un archivo llamado:
procesos.csv
Ejemplo:
id,llegada,duracion
P1,0,5
P2,2,3
P3,3,8
P4,6,2

Cada fila representa un proceso. El quantum **no** se incluye en el archivo: se recibe como parámetro.

Significado de las columnas

Columna	Significado	Ejemplo
id	Identificador único del proceso.	P1 P2 Limpieza01 ModeloA
llegada	Unidad de tiempo en la cual el proceso ingresa al sistema.	Debe ser un número entero mayor o igual que cero.
duracion	Cantidad de unidades de tiempo de CPU que necesita el proceso para terminar.	Debe ser un número entero mayor que cero.

7. Restricciones sobre los datos

El programa deberá validar al menos las siguientes situaciones:

- identificadores vacíos;
- identificadores duplicados;
- tiempos de llegada negativos;
- duraciones iguales o menores que cero;
- valores que no puedan convertirse a números enteros;
- archivo inexistente;
- archivo vacío;
- filas incompletas;
- política no reconocida;
- quantum que no sea un número entero mayor que cero (cuando la política es rr).

Cuando se encuentre un error, el programa deberá mostrar un mensaje comprensible.

No es necesario intentar reparar automáticamente el archivo.

8. Avance del tiempo

La simulación deberá comenzar en:
tiempo = 0
El tiempo avanzará en unidades enteras: 0, 1, 2, 3, 4, 5...

En cada unidad de tiempo el simulador deberá considerar:

1. cuáles procesos llegan en ese instante;
2. cuáles deben ingresar a la cola;

3. si el proceso que está en el CPU terminó;
4. si el proceso que está en el CPU agotó su quantum (solo en Round Robin);
5. si el CPU está libre y cuál proceso debe comenzar o continuar su ejecución;
6. cuánto tiempo de ejecución le queda al proceso actual;
7. si el CPU permanece ocioso porque todavía no hay procesos listos.

Puede avanzar el reloj unidad por unidad o saltar directamente entre eventos, siempre que el resultado sea idéntico. Se recomienda la primera opción por ser más fácil de depurar.

9. Políticas de ejecución

9.1 FIFO

Debe funcionar exactamente igual que en la Tarea 1: el primer proceso que entra a la cola es el primero en salir, y una vez que obtiene el CPU lo conserva hasta terminar. Si varios procesos llegan en el mismo instante, respetan el orden en que aparecen en el archivo de entrada.

9.2 Round Robin

Las siguientes reglas deben cumplirse al pie de la letra, para que todas las soluciones produzcan resultados idénticos con un mismo archivo de entrada y un mismo quantum:

1. Los procesos que llegan entran al final de la cola de listos. Si varios llegan en el mismo instante, respetan el orden en que aparecen en el archivo.
2. Cuando el CPU está libre, se asigna al proceso que está al frente de la cola.
3. Un proceso puede usar el CPU un máximo de Q unidades de tiempo consecutivas.
4. Si el proceso termina antes de agotar su quantum, o justo al agotarlo, pasa a Terminado y libera el CPU. El siguiente proceso de la cola comienza en ese mismo instante.
5. Si el quantum se agota y al proceso todavía le queda tiempo, es expropiado: pasa a estado Listo y se coloca al final de la cola.
6. **Regla de desempate:** si en el mismo instante en que un proceso agota su quantum llegan procesos nuevos, los procesos nuevos entran primero a la cola y después entra el proceso expropiado.
7. Si al agotarse el quantum la cola está vacía, el proceso continúa ejecutándose con un nuevo quantum. En ese caso no hay expropiación ni cambio de contexto.

8. Si no hay procesos listos ni ejecutándose, pero aún faltan procesos por llegar, el CPU permanece ocioso y el tiempo sigue avanzando.
9. El cambio de contexto no consume tiempo (cuesta 0 unidades). Se contabiliza cada vez que el CPU se asigna a un proceso distinto del último que lo utilizó; la primera asignación del CPU no cuenta.

10. Ejemplo conceptual

Considere un quantum de 2 y los siguientes procesos:
P1 llega en 0 y dura 5
P2 llega en 2 y dura 3
P3 llega en 3 y dura 4

El CPU se asignaría de la siguiente manera:
P1 (0-2) → P2 (2-4) → P1 (4-6) → P3 (6-8) → P2 (8-9) → P1 (9-10) → P3 (10-12)

Estado del sistema cada vez que se asigna el CPU (la cola se lee de izquierda a derecha; el frente es el proceso de la izquierda):

Tiempo	Proceso en CPU	Cola de listos	Observación
0	P1	(vacía)	Llega P1 y obtiene el CPU.
2	P2	P1	P1 agota su quantum (le quedan 3). Llega P2 en ese mismo instante: entra a la cola antes que P1.
4	P1	P3, P2	P2 agota su quantum (le queda 1). P3 había llegado en 3.
6	P3	P2, P1	P1 agota su quantum (le queda 1).
8	P2	P1, P3	P3 agota su quantum (le quedan 2).
9	P1	P3	P2 termina.
10	P3	(vacía)	P1 termina.
12	—	(vacía)	P3 termina. Fin de la simulación.

Con estos datos, los resultados serían:

Proceso	Llegada	Duración	Inicio	Fin	Espera	Retorno	Respuesta
P1	0	5	0	10	5	10	0
P2	2	3	2	9	4	7	0
P3	3	4	6	12	5	9	3

Espera promedio: 4.67; retorno promedio: 8.67; respuesta promedio: 1.00.

Compare con lo que ocurre con FIFO para los mismos procesos (Tarea 1): espera promedio 2.67, retorno promedio 6.67 y respuesta promedio 2.67. Round Robin mejora el tiempo de respuesta, pero empeora la espera y el retorno promedio. Entender este compromiso es parte central del análisis que se pide en esta tarea.

La representación exacta de los mensajes queda a criterio del estudiante, siempre que el comportamiento pueda entenderse con claridad.

11. Estados mínimos de los procesos

Se mantienen los estados de la Tarea 1, pero ahora un proceso puede volver al estado Listo después de haber estado Ejecutando.

Nuevo

El proceso todavía no ha llegado al sistema.

Listo

El proceso llegó (o fue expropiado) y se encuentra esperando por el procesador.

Ejecutando

El proceso está utilizando el CPU.

Terminado

El proceso completó toda su ejecución.

Transiciones válidas:

- Nuevo → Listo: el proceso llega al sistema;
- Listo → Ejecutando: se le asigna el CPU;
- Ejecutando → Terminado: completó su duración;
- Ejecutando → Listo: agotó su quantum y todavía le queda tiempo (**transición nueva respecto a la Tarea 1**).

12. Datos que deberá almacenar cada proceso

Como mínimo, cada proceso deberá manejar:

- identificador
- tiempo de llegada (llegada en el archivo)
- duración (duración en el archivo)
- **tiempo restante** (cuánto le falta por ejecutar; empieza igual a la duración y disminuye mientras usa el CPU)
- tiempo de inicio (momento en que obtiene el CPU **por primera vez**)
- tiempo de finalización (momento en que termina)

- tiempo de espera (tiempo total que pasó en la cola de listos, sumando todas las veces que esperó)
- tiempo de retorno y tiempo de respuesta: ver siguiente sección
- **cantidad de expropiaciones** (veces que perdió el CPU sin haber terminado)

13. Cálculo de métricas

El simulador deberá calcular para cada proceso las siguientes métricas.

Tiempo de retorno

También conocido como *turnaround time*. Se calcula igual que en la Tarea 1:
tiempo_retorno = tiempo_finalizacion - tiempo_llegada

Tiempo de espera

En la Tarea 1, como los procesos no eran interrumpidos, la espera ocurría solamente antes de la primera ejecución. Ahora un proceso espera cada vez que vuelve a la cola, por lo que la fórmula cambia:
tiempo_espera = tiempo_retorno - duracion
Representa todo el tiempo que el proceso estuvo en el sistema sin usar el CPU.

Tiempo de respuesta

Es lo que en la Tarea 1 se calculaba como “espera”. Mide cuánto tardó el proceso en obtener el CPU por primera vez:
tiempo_respuesta = tiempo_inicio - tiempo_llegada

Con FIFO, el tiempo de espera y el tiempo de respuesta de un proceso son iguales. Con Round Robin generalmente no lo son. Puede usar esto como una verificación de su implementación.

14. Métricas globales

Al terminar la simulación deberán mostrarse al menos:

- política utilizada y, si es rr, el quantum;
- cantidad total de procesos;
- tiempo total de simulación (desde cero hasta que terminó el último proceso del archivo);
- tiempo promedio de espera;
- tiempo promedio de retorno;
- **tiempo promedio de respuesta;**

- proceso con mayor tiempo de espera;
- proceso con menor tiempo de espera;
- **utilización del CPU:** (unidades de tiempo en que el CPU estuvo ocupado / tiempo total) × 100;
- **cantidad de cambios de contexto.**

En caso de empate en mayor o menor espera, se muestra el primer proceso en el orden del archivo.

15. Salida en pantalla

Al finalizar la ejecución deberá presentarse un resumen similar al siguiente (ejemplo con el caso de la sección 10):

==========
RESULTADO MINIOS
Política: Round Robin (quantum = 2)
==========

Proceso	Llegada	Inicio	Fin	Espera	Retorno	Respuesta
P1	0	0	10	5	10	0
P2	2	2	9	4	7	0
P3	3	6	12	5	9	3

Procesos ejecutados: 3
Tiempo total: 12
Espera promedio: 4.67
Retorno promedio: 8.67
Respuesta promedio: 1.00
Mayor espera: P1 (5)
Menor espera: P2 (4)
Utilización del CPU: 100.0%
Cambios de contexto: 6

No es necesario utilizar exactamente este formato (la tabla puede imprimirse como texto alineado). Sin embargo, la información debe presentarse de manera ordenada y comprensible.

16. Archivo de resultados

Además de mostrar resultados en pantalla, el programa deberá generar un archivo: resultados.csv

Con una estructura semejante a (mismo ejemplo de la sección 10):
id,llegada,duracion,inicio,fin,espera,retorno,respuesta,expropiaciones
P1,0,5,0,10,5,10,0,2
P2,2,3,2,9,4,7,0,1
P3,3,4,6,12,5,9,3,1

Las primeras siete columnas conservan el significado de la Tarea 1 (con *inicio* igual a la primera vez que el proceso obtiene el CPU y *espera* calculada con la nueva fórmula). Este archivo se utilizará posteriormente para análisis de datos. Las filas deben ir en el mismo orden del archivo de entrada.

17. Registro de eventos

El programa deberá generar adicionalmente un archivo:
eventos.txt
En él deberá quedar registrado lo ocurrido durante la simulación, incluyendo ahora las expropiaciones. Por ejemplo (mismo caso de la sección 10):
[0] P1 llega al sistema
[0] P1 inicia ejecución
[2] P2 llega al sistema
[2] P2 entra a la cola
[2] P1 agota su quantum (restante=3)
[2] P1 vuelve a la cola
[2] P2 inicia ejecución
[3] P3 llega al sistema
[3] P3 entra a la cola
[4] P2 agota su quantum (restante=1)
[4] P2 vuelve a la cola
[4] P1 retoma ejecución
[6] P1 agota su quantum (restante=1)
[6] P1 vuelve a la cola
[6] P3 inicia ejecución
[8] P3 agota su quantum (restante=2)
[8] P3 vuelve a la cola
[8] P2 retoma ejecución
[9] P2 termina
[9] P1 retoma ejecución
[10] P1 termina

Este archivo funcionará como una especie de registro o *log* del simulador. Si en algún momento el CPU queda ocioso, también debe quedar registrado.

18. Casos de prueba

Para verificar su implementación, el programa debe reproducir los siguientes resultados con el archivo procesos.csv de la sección 4 (P1: 0, 5; P2: 2, 3; P3: 3, 8; P4: 6, 2), política rr y quantum 3:

Proceso	Inicio	Fin	Espera	Retorno	Respuesta	Expropiaciones
P1	0	11	6	11	0	1
P2	3	6	1	4	1	0
P3	6	18	7	15	3	2
P4	11	13	5	7	5	0

Tiempo total: 18; espera promedio: 4.75; retorno promedio: 9.25; respuesta promedio: 2.25; utilización del CPU: 100.0%; cambios de contexto: 5.

Además, el estudiante deberá crear **al menos cuatro archivos de prueba propios** (con nombres descriptivos) que cubran, entre otros, los siguientes casos:

- un solo proceso;
- varios procesos con el mismo instante de llegada;
- un proceso que llega exactamente cuando otro agota su quantum (regla de desempate);
- un hueco de tiempo en el que el CPU queda ocioso (por ejemplo, P1 llega en 0 y dura 2; P2 llega en 10 y dura 3);
- quantum igual a 1;
- quantum mayor o igual que la duración del proceso más largo: el resultado debe ser idéntico al de FIFO;
- archivos inválidos para comprobar los mensajes de error.

19. Análisis comparativo

Utilice su simulador para comparar FIFO y Round Robin y entregue un informe breve (máximo 2 páginas, en PDF o Word). El informe debe incluir:

1. Una tabla comparativa con los resultados globales de FIFO y de Round Robin con al menos tres valores de quantum distintos (por ejemplo 1, 3 y un valor mayor o igual a la duración más larga), ejecutados sobre procesos.csv.

2. La misma comparación sobre un archivo propio de **al menos 6 procesos**, que mezcle procesos largos y cortos y que incluya llegadas en momentos diferentes.
3. Las respuestas, con base en los datos obtenidos, a las siguientes preguntas:
   - ¿Qué política produce el menor tiempo de respuesta promedio? ¿Cuál produce el menor tiempo de espera promedio? ¿Por qué no son siempre la misma?
   - ¿Qué ocurre con los cambios de contexto cuando el quantum es muy pequeño? ¿Y cuando es muy grande?
   - ¿Se observa el efecto convoy en FIFO? Indique un proceso concreto de sus datos que lo ilustre.
   - Si cada cambio de contexto costara 1 unidad de tiempo, ¿cómo cambiaría la utilización útil del CPU con quantum 1? (No es necesario implementarlo; basta con razonarlo.)
   - ¿Qué valor de quantum recomendaría para sus datos y por qué?

Plantilla sugerida para la tabla comparativa:

Política	Quantum	Espera prom.	Retorno prom.	Respuesta prom.	Cambios de contexto	Utilización CPU
FIFO	—					
RR	1					
RR	3					
RR						

Nota: como el programa escribe siempre resultados.csv y eventos.txt, copie o renombre esos archivos después de cada corrida si desea conservarlos.

20. Restricción tecnológica

El programa deberá desarrollarse utilizando exclusivamente **Python y su biblioteca estándar.**

Por lo tanto, no se permite utilizar bibliotecas externas. NADA QUE REQUIERA PIP INSTALL.

No se permite, por ejemplo:
pandas numpy matplotlib simpy polars scipy

Sí pueden utilizarse módulos pertenecientes a la biblioteca estándar, tales como:

csv, collections, statistics, dataclasses, pathlib, random, sys, argparse, unittest. Úselas a discreción. Para la cola de procesos listos se recomienda collections.deque.

21. Organización esperada del programa

El código deberá separarse en funciones. No se aceptará como solución adecuada un programa completo escrito únicamente dentro de un bloque secuencial.

Se recomienda contar con funciones semejantes a:

leer_procesos(...)
validar_procesos(...)
leer_parametros(...)
simular_fifo(...)
simular_round_robin(...)
calcular_metricas(...)
guardar_resultados(...)
guardar_eventos(...)
mostrar_resumen(...)
main()

Los nombres anteriores son solamente ejemplos. **El estudiante puede definir su propia organización.** Se espera, eso sí, que la lectura, la validación, la simulación y la salida de resultados estén claramente separadas, y que las métricas se calculen con una función que sirva para ambas políticas.

22. Entregables

Debe entregar, comprimidos en un solo archivo .zip y por el medio indicado en el curso:

- minios.py;
- procesos.csv;
- los archivos de prueba propios (mínimo cuatro) y el archivo de al menos 8 procesos usado en el análisis;
- un resultados.csv y un eventos.txt de ejemplo, generados con procesos.csv, política rr y quantum 3;
- el informe del análisis comparativo.

23. Cronograma sugerido

Semana	Fechas	Avance esperado
1	12 al 19 de octubre	Corregir lo pendiente de la Tarea 1. Agregar parámetros de línea de comandos. Separar simular_fifo. Implementar Round Robin con cola de listos y tiempo restante. Reproducir a mano y con el programa el ejemplo de la sección 10. Generar eventos.txt.
2	19 al 26 de octubre	Calcular las nuevas métricas y archivos de salida. Completar las validaciones. Crear los casos de prueba y comparar contra la sección 18. Ejecutar los experimentos y redactar el informe. Revisión final y entrega.

24. Evaluación

Criterio	Porcentaje
Funcionamiento correcto de Round Robin (reglas de la sección 9.2)	30%
FIFO se mantiene funcionando como en la Tarea 1	5%
Validaciones y mensajes de error	10%
Métricas, salida en pantalla, resultados.csv y eventos.txt	20%
Organización del código en funciones y claridad	10%
Casos de prueba propios	10%
Informe de análisis comparativo	15%
Total	**100%**

Puntos extra (hasta 5%): una opción --comparar que muestre FIFO y Round Robin lado a lado, o una opción --costo-cambio para simular un costo de cambio de contexto mayor que cero.